Experiment 20: Probabilistic PCA and Reconstruction Analysis

Question

Problem Statement: A sensor network in a smart factory produces readings with random noise and occasional missing values. Engineers want a model that accounts for noise explicitly. Multivariate sensor readings (temperature, vibration, pressure, current and so on) with added Gaussian noise. The objective is to apply probabilistic PCA and compare its reconstruction with standard PCA. Use NumPy, Scikit-learn, Matplotlib.

Tasks:

1. Explain the PPCA generative model $x = Wz + \mu + \varepsilon$.
2. Fit PPCA (Scikit-learn PCA with its noise-variance estimate, or an EM implementation in NumPy).
3. Estimate the noise variance σ^2.
4. Reconstruct the data and compute reconstruction error against component count.
5. Generate new synthetic sensor samples from the fitted model.

Code

```
"""Experiment 20: Probabilistic PCA and Reconstruction Analysis (noisy sensor data)"""
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

np.random.seed(42)

# ---------------- Data: 8 sensors driven by 3 hidden factors + Gaussian noise ----------------
sensors = ["Temperature", "Vibration", "Pressure", "Current", "Speed", "Voltage", "Flow", "Humidity"]
n, d, q_true, sigma_true = 600, 8, 3, 0.5
mu_true = np.array([60, 2.0, 101, 12, 1500, 230, 40, 55], dtype=float)
W_true = np.random.randn(d, q_true) * 2.0
Z = np.random.randn(n, q_true)
clean = Z @ W_true.T + mu_true                                  # noise-free signal
X = clean + np.random.normal(0, sigma_true, (n, d))             # observed readings
print(f"True latent dimension q = {q_true}, true noise variance sigma^2 = {sigma_true**2:.3f}")

# ---------------- Task 1: the PPCA generative model ----------------
print("""
PPCA generative model:   x = W z + mu + eps
    z   ~ N(0, I_q)            latent variables (q < d)
    eps ~ N(0, sigma^2 I_d)    isotropic Gaussian noise
    => x ~ N(mu, W W^T + sigma^2 I)
Maximum-likelihood solution (Tipping & Bishop):
    sigma^2 = mean of the (d - q) smallest eigenvalues of the sample covariance
    W       = U_q (Lambda_q - sigma^2 I)^(1/2)
Ordinary PCA is the limit sigma^2 -> 0.""")

# ---------------- Task 2 & 3: fit PPCA, estimate sigma^2 ----------------
mu_hat = X.mean(axis=0)
Xc = X - mu_hat
S = Xc.T @ Xc / n                                               # ML covariance (divide by n)
evals, evecs = np.linalg.eigh(S)
order = np.argsort(evals)[::-1]; evals, evecs = evals[order], evecs[:, order]

def fit_ppca(q):
    sigma2 = evals[q:].mean()
    W = evecs[:, :q] @ np.diag(np.sqrt(np.maximum(evals[:q] - sigma2, 0)))
    return W, sigma2

W3, sigma2_3 = fit_ppca(q_true)
skl = PCA(n_components=q_true).fit(X)
print(f"\nEstimated sigma^2 (closed-form PPCA)        : {sigma2_3:.4f}")
print(f"Estimated sigma^2 (sklearn PCA.noise_variance_): {skl.noise_variance_:.4f}")
print(f"True sigma^2                                : {sigma_true**2:.4f}")

# ---------------- Task 4: reconstruction error vs number of components ----------------
def pca_reconstruct(k):
```

```
    V = evecs[:, :k]
    return (Xc @ V) @ V.T + mu_hat

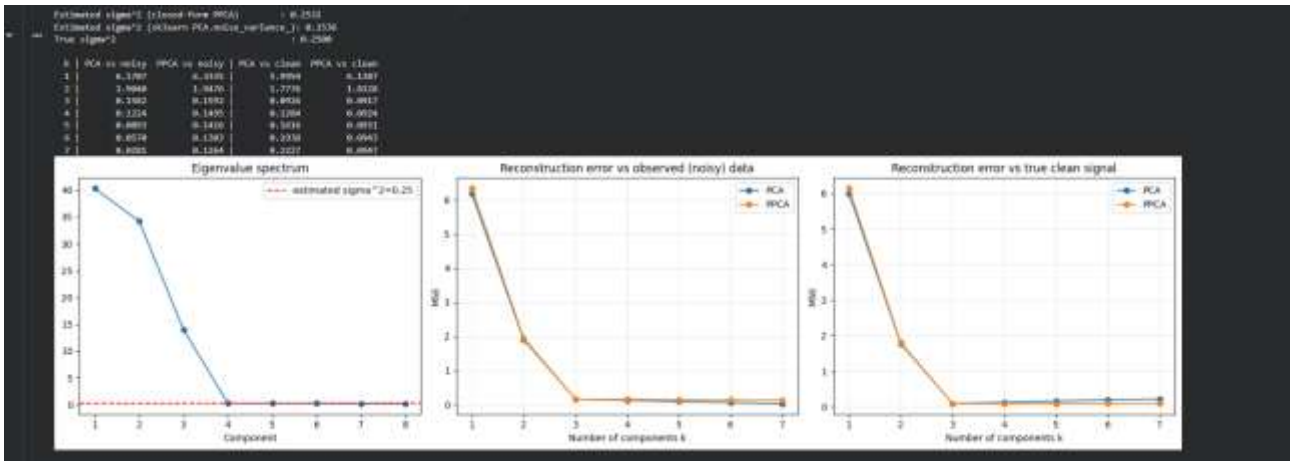
def ppca_reconstruct(k):
    W, s2 = fit_ppca(k)
    M = W.T @ W + s2 * np.eye(k)
    Ez = Xc @ W @ np.linalg.inv(M)                              # posterior mean E[z | x] = M^-1 W^T (x -
mu)
    return Ez @ W.T + mu_hat

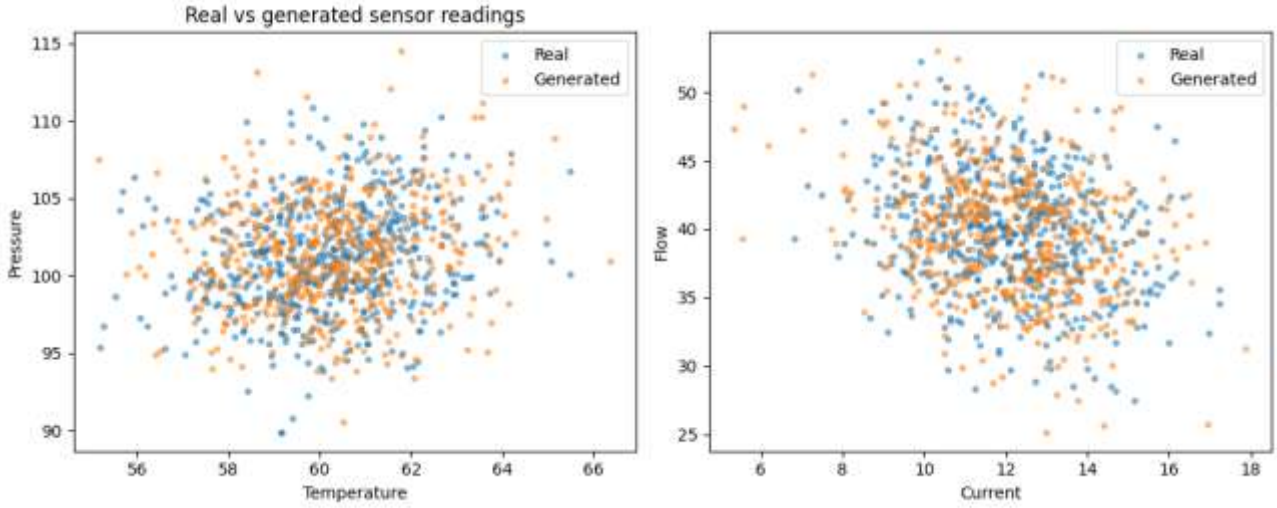
ks = list(range(1, d))
err = {name: {"noisy": [], "clean": []} for name in ["PCA", "PPCA"]}
for k in ks:
    for name, fn in [("PCA", pca_reconstruct), ("PPCA", ppca_reconstruct)]:
        rec = fn(k)
        err[name]["noisy"].append(np.mean((rec - X) ** 2))        # error against observed data
        err[name]["clean"].append(np.mean((rec - clean) ** 2))    # error against the true noise-free signal
print(f"\n{'k':>3} | {'PCA vs noisy':>12} {'PPCA vs noisy':>14} | {'PCA vs clean':>12} {'PPCA vs
clean':>14}")
for i, k in enumerate(ks):
    print(f"{k:3d} | {err['PCA']['noisy'][i]:12.4f} {err['PPCA']['noisy'][i]:14.4f} | "
          f"{err['PCA']['clean'][i]:12.4f} {err['PPCA']['clean'][i]:14.4f}")

fig, ax = plt.subplots(1, 3, figsize=(17, 4.5))
ax[0].plot(range(1, d + 1), evals, "o-"); ax[0].axhline(sigma2_3, color="r", ls="--", label=f"estimated
sigma^2={sigma2_3:.2f}")
ax[0].set_title("Eigenvalue spectrum"); ax[0].set_xlabel("Component"); ax[0].legend()
for name in err:
    ax[1].plot(ks, err[name]["noisy"], "o-", label=name); ax[2].plot(ks, err[name]["clean"], "o-",
label=name)
ax[1].set_title("Reconstruction error vs observed (noisy) data"); ax[2].set_title("Reconstruction error vs
true clean signal")
for a in ax[1:]:
    a.set_xlabel("Number of components k"); a.set_ylabel("MSE"); a.legend(); a.grid(alpha=0.3)
plt.tight_layout(); plt.show()

# ---------------- Task 5: generate new synthetic sensor samples ----------------
Z_new = np.random.randn(500, q_true)
X_new = Z_new @ W3.T + mu_hat + np.random.normal(0, np.sqrt(sigma2_3), (500, d))     # x = Wz + mu + eps
cov_gap = np.linalg.norm(np.cov(X_new, rowvar=False) - np.cov(X, rowvar=False)) / np.linalg.norm(np.cov(X,
rowvar=False))
print(f"\nGenerated {len(X_new)} new samples; relative covariance difference vs real data = {cov_gap:.3f}")
fig, ax = plt.subplots(1, 2, figsize=(11, 4.5))
ax[0].scatter(X[:, 0], X[:, 2], s=10, alpha=0.5, label="Real"); ax[0].scatter(X_new[:, 0], X_new[:, 2], s=10,
alpha=0.5, label="Generated")
ax[0].set_xlabel(sensors[0]); ax[0].set_ylabel(sensors[2]); ax[0].legend(); ax[0].set_title("Real vs
generated sensor readings")
ax[1].scatter(X[:, 3], X[:, 6], s=10, alpha=0.5, label="Real"); ax[1].scatter(X_new[:, 3], X_new[:, 6], s=10,
alpha=0.5, label="Generated")
ax[1].set_xlabel(sensors[3]); ax[1].set_ylabel(sensors[6]); ax[1].legend()
plt.tight_layout(); plt.show()
```

Output


Eigenvalue spectrum
estimated sigma^2=0.25
Component
Reconstruction error vs observed (noisy) data
PCA
PPCA
MSE
Number of components k
Reconstruction error vs true clean signal
PCA
PPCA
MSE
Number of components k


Real vs generated sensor readings
Real
Generated
Pressure
Temperature
Real
Generated
Flow
Current